

SD Basic - Dynamic Arrays

Fields, values and subvalues - reading, changing and searching the structure that holds every SD record.

A dynamic array is a string with structure in it. Three mark characters divide it into **fields**, **values** and **subvalues**, and every record SD reads or writes is one. This page covers the statements and functions that work on that structure.

The functions that treat the same string as plain text are in SD Basic - String Functions; the two pages overlap where a function is used both ways, and each says so.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

In the examples below, `^` stands for a field mark, `|` for a value mark and `\` for a subvalue mark.

The marks

	NAME	CONSTANT	CODE
<code>^</code>	field mark	<code>@fm</code>	254
<code> </code>	value mark	<code>@vm</code>	253
<code>\</code>	subvalue mark	<code>@sm</code>	252

They nest: a field contains values, a value contains subvalues. There is a fourth, `@tm` (text mark, 251), used by `fmt()` and by some conversions, and `@im` (item mark, 255), which SD uses internally.

```
a = 'f1' : @fm : 'v1' : @vm : 'v2' : @fm : 'f3'
```

gives `f1^v1|v2^f3` — three fields, the second of which holds two values.

The marks are ordinary characters and they are counted. `len(a)` is `11` for the array above — eight data characters and three marks. Everything in SD Basic - String Functions sees them.

```
dcount(string, delimiter)
```

counts the pieces a delimiter divides a string into. On the array above, `dcount(a, @fm)` is `3` and `dcount(a<2>, @vm)` is `2`.

`dcount()` of an empty string is `0`, not `1`. That is what makes `for i = 1 to dcount(rec, @fm)` correct — an empty record does nothing.

Reading a piece

```
array<field {, value {, subvalue}}>  
extract(array, field {, value {, subvalue}})
```

The angle-bracket form and `extract()` are the same operation.

EXPRESSION ON <code>F1^V1\ V2^F3</code>	RESULT
<code>a<2></code>	<code>v1\ v2</code> — the whole field, marks and all
<code>a<2,2></code>	<code>v2</code>
<code>a<2,1,1></code>	<code>v1</code>
<code>extract(a, 2, 1)</code>	<code>v1</code>
<code>a<9></code>	<i>empty</i>

Reading past the end is not an error. `a<9>` on a three-field array returns the null string, exactly as a genuinely empty field would. **There is no way to tell "field 9 is empty" from "there is no field 9"** — use `dcount()` if the difference matters.

Asking for a value of a field that has none returns the whole field: `a<1,1>` is `f1`.

Changing a piece

```
array<field {, value {, subvalue}}> = expression  
replace(array, field, value, subvalue, expression)  
insert(array, field, value, subvalue, expression)  
delete(array, field {, value {, subvalue}})
```

EXPRESSION ON <code>f1^v1\ v2^f3</code>	RESULT
<code>b<2,2> = 'NEW'</code>	<code>f1^v1\ NEW^f3</code>
<code>replace(a, 2, 2, 0, 'R')</code>	<code>f1^v1\ R^f3</code>
<code>insert(a, 2, 2, 0, 'I')</code>	<code>f1^v1\ I\ v2^f3</code> — pushes the rest along
<code>delete(a, 2, 1)</code>	<code>f1^v2^f3</code> — removes one value
<code>delete(a, 2)</code>	<code>f1^f3</code> — removes the whole field

The assignment form is what almost all code uses. `replace()` and `insert()` exist for when you need the result as an expression rather than assigned back.

`replace()` and `insert()` count their arguments greedily, and the short forms need a semicolon.

`replace(a, 2, 2, 'R')` **does not compile** — the parser takes `2` as the value, `'R'` as the subvalue, and then demands the replacement string it never got. Either give all five arguments, or separate the final one with a semicolon.

CALL	RESULT
<code>replace(a, 2, 2, 0, 'R')</code>	<code>f1^v1\ R^f3</code>
<code>replace(a, 2; 'S')</code>	<code>f1^S^f3</code> — replaces the whole field
<code>replace(a, 2, 2; 'T')</code>	<code>f1^v1\ T^f3</code>
<code>replace(a, 2, 2, 'R')</code>	compile error

APPENDING

A subscript of `-1` appends:

	RESULT
<code>b<-1> = 'APP'</code>	<code>f1^v1\ v2^f3^APP</code> — a new field at the end
<code>b<2,-1> = 'AV'</code>	<code>f1^v1\ v2\ AV^f3</code> — a new value in field 2

Writing past the end pads with empty fields rather than failing. `insert(a, 9, 0, 0, 'FAR')` on a three-field array gives `f1^v1\|v2^f3^^^^^FAR` — five empty fields appear to fill the gap. A loop with an off-by-one index therefore grows the record silently instead of raising anything.

THE STATEMENT FORMS

```
del array<field {, value {, subvalue}}>
ins string before array<field {, value {, subvalue}}>
```

`del` and `ins` change the variable in place. `del b<2>` gives `f1^f3`, and `ins 'X' before b<2>` gives `f1^X^v1|v2^f3`.

Searching

LOCATE

```
locate expression in array<field {, value {, subvalue}}> {by order} setting variable
    then statements
else statements

locate(expression, array, field {, value {, subvalue}})
```

`locate` finds a whole element and reports its position. When it does not find one, `setting` receives **the position where it would be inserted** to keep the order — which is what makes it the standard way to add to a sorted list.

The number of subscripts chooses the level searched. this is the thing people get wrong. With `vals` holding `apple|cherry|damson` as three values in one field:

STATEMENT	RESULT
<code>locate 'cherry' in vals<1> setting p</code>	miss , insert at 2
<code>locate 'cherry' in vals<1,1> setting p</code>	found at 2

`array<f>` searches the **fields**, starting at field *f*. `array<f,v>` searches the **values of field f**, starting at value *v*. `array<f,v,s>` searches subvalues. So a single subscript on an array of values finds nothing, silently, and reports an insertion point — which looks like a working search right up until the data has two fields.

The statement form requires the angle brackets. `locate 'x' in vals setting p` does not compile: *"Field reference not found where expected"*.

The starting subscript really does start there. With `vals<1,2>`, `locate 'apple'` — which is value 1 — **misses**, and reports an insertion point of 4.

The function form takes the position as arguments instead, and returns the position or zero:

CALL	RESULT
<code>locate('cherry', vals, 1)</code>	<code>0</code> — searched fields
<code>locate('cherry', vals, 1, 1)</code>	<code>2</code> — searched values
<code>locate('zebra', vals, 1, 1)</code>	<code>0</code>

ORDER CODES

Without `by`, `locate` scans the whole list and the insertion point is the end. With `by`, it assumes the list is already sorted that way, stops as soon as it passes the place the value would go, and reports that place.

CODE	ORDER
<code>al</code>	ascending, left-justified — ordinary text order
<code>ar</code>	ascending, right-justified — numbers compare as numbers
<code>dl</code>	descending, left-justified
<code>dr</code>	descending, right-justified

For `apple|cherry|damson`:

SEARCH	INSERT AT
<code>'aardvark' by 'al'</code>	<code>1</code>
<code>'banana' by 'al'</code>	<code>2</code>
<code>'zebra' by 'al'</code>	<code>4</code>

And the justification is not a detail. For the values `2`, `10`, `30`, looking for `9`:

	INSERT AT
<code>by 'al'</code>	<code>4</code> — as text, <code>9</code> sorts after <code>30</code>
<code>by 'ar'</code>	<code>2</code> — as numbers, between <code>2</code> and <code>10</code>

Use `ar` for anything numeric. `al` on numbers produces a list that looks sorted in short test data and comes apart at ten items.

`by` on a list that is *not* actually sorted that way stops early and reports a miss for a value that is present.

```
find string in array {, occurrence} setting field {, value {, subvalue}}
findstr string in array {, occurrence} setting field {, value {, subvalue}}
```

`find` matches a **whole element**; `findstr` matches **any element containing the string**. Both report the position as separate field, value and subvalue numbers rather than one index.

For `p^q|r`:

	RESULT
<code>find 'r' in d setting f, v, s</code>	field 2, value 2, subvalue 1
<code>findstr 'q' in d setting f, v</code>	field 2, value 1

Continued in

SD Basic - Operating on Whole Arrays — walking an array, mark levels, the s functions, logical functions, sorting and mark mapping.